

# פרק 3

## טיפוסי התייחסות – Reference Types

# טיפוסים ערך – Value Types

שני סוגי טיפוסים –

טיפוסים ערך וטיפוסים התייחסות

Value Types / Reference Types

טיפוסים ערך הם בדיוק כמו הטיפוסים ב- C++ והפרימיטיביים ב- Java

משתנה מסוג טיפוס ערך מחזיק את הערך בעצמו

מיוצר במחסנית של המתודה (stack)

מועתק בהשמה

מועתק כשנשלח למתודה (By Value)

להבדיל מ- C++: חובה לאתחל משתנה מקומי לפני השימוש בו

טיפוסים ערך הם Structs/Enums

כל הטיפוסים הבסיסיים (פרימיטיביים) הם Structs

נורשים מ- System.ValueType (שירש מ- object)

# טיפוסי התייחסות – Reference Types

מוקצים דינאמית על ה-Heap

מחזיקים התייחסות (reference) לאובייקט עצמו  
(בפועל כמו מצביע/כתובת)

זמן המחיה מנוהל ע"י ה-Garbage Collector

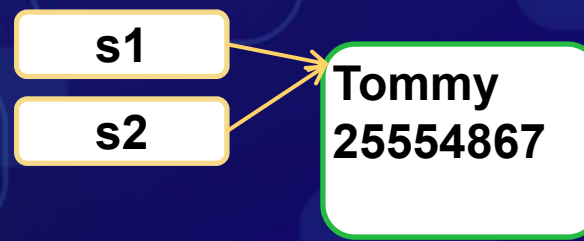
בהשמה מועתקת ההתייחסות ולא האובייקט עצמו

בשליחה למתודה מועברת ההתייחסות לאובייקט ולא  
האובייקט עצמו

Any class, Any array, string, object, delegate

ניתן גם להעביר כ- ref או כ- out (כמו לשלוח מצביע למצביע)

```
class Student {  
    string m_Name = "Tommy";  
    int m_Id = 25554867;  
}  
  
Student s1 = new Student();  
Student s2 = s1;
```

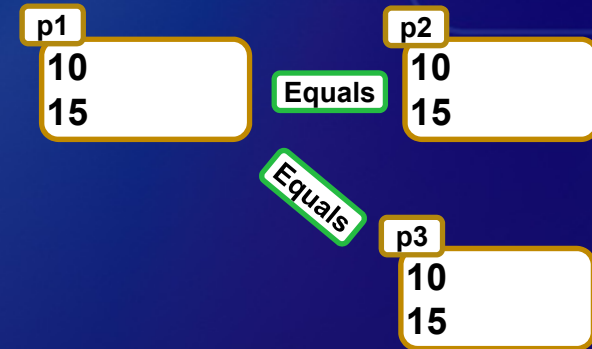


# השוואה והעתקה

■ בהשוואה והעתקה של טיפוסים ערך (Value Type), הפעולה מתבצעת על האובייקט עצמו:

```
struct Point {  
    int m_X;  
    int m_Y;  
}
```

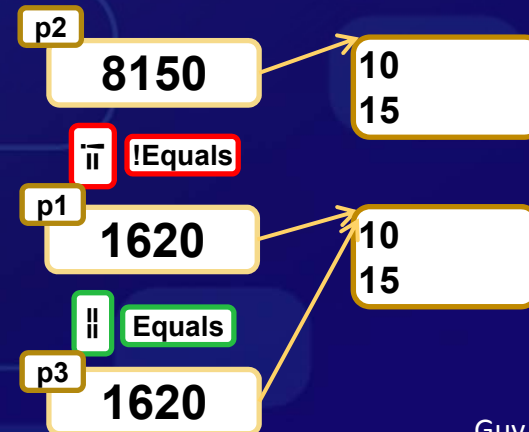
```
1. Point p1 = new Point(10, 15);  
2. Point p2 = new Point(10, 15);  
3. Point p3 = p2;  
4. bool equals = (p1 == p2); // compile error!!  
5. bool equals = (p2 == p3); // compile error!!  
6. bool equals = p1.Equals(p3); // true!!  
7. bool equals = p1.Equals(p2); // true!!
```



■ בהשוואה והעתקה של טיפוסים התייחסות (Reference), תתבצע על ה"מתייחסים" (כלומר שני "מתייחסים" נחשבים שווים, אם הם מתייחסים לאותו אובייקט בזיכרון):

```
class Point {  
    int m_X;  
    int m_Y;  
}
```

```
1. Point p1 = new Point(10, 15);  
2. Point p2 = new Point(10, 15);  
3. Point p3 = p1;  
4. bool equals = (p1 == p2); // false  
5. bool equals = (p1 == p3); // true  
6. bool equals = (p2 == p3); // false  
7. bool equals = p1.Equals(p2); // false  
8. bool equals = p1.Equals(p3); // true
```



## סוגים שונים של השוואת ערכים

ניתן לבצע העמסה של האופרטור ==

ניתן לבצע העמסה של המתודה Equals

ניתן להשתמש במתודה הסטטית –

`Object.ReferenceEquals(object a, object b)`

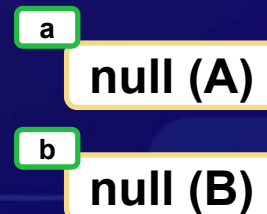
כדי להשוות מתייחסים בלי להסתכן שבוצעה העמסת  
אופרטור השוואה



## איתחולים ו- null

- ב- C# חובה לאתחל משתנים לוקאליים לפני השימוש
- משתנים לוקאליים לא מאותחלים לערכי ברירת מחדל
- משתנה ערך (Value Type) חובה לאתחל בערך אמיתי
- משתנה התייחסות (reference) ניתן לאתחל ע"י יצירת אובייקט וקבלת התייחסות או ע"י הערך null.
- ניתן לשלוח למתודה משתנה שמכיל את הערך null אולם לא ניתן להשתמש בו / להפעיל אותו לפני הקצאה של אובייקט אמיתי
- שני משתנים מטיפוסים שונים שמכילים את הערך null אינם נחשבים שווים!

```
class A {}  
class B {}  
  
A a = null;  
B b = null;  
bool equals = (a == b); // error
```



## nullable

לעיתים נרצה את האפשרות לייצג את הערך "כלום" גם למשתנים של טיפוסי ערך (Value Types).

נשתמש במשתנה המקביל מסוג nullable value type ע"י הוספת סימן שאלה (?) או שימוש במבנה הגנרי.

דוגמאות תחביר:

```
1. int? nullableInt = null; // an int that can be assigned with null
2. Nullable<int> nullableInt2 = 15; // can assign a simple int
3. if(nullableInt != null){ // one way to check if has a value
4.     if(nullableInt2.HasValue){ // another one way to check if has a value
5.         int val = nullableInt2.Value + nullableInt.GetValueOrDefault();
6.     }
7. }
```

האופרטור "??":

```
1. int y = x ?? 25; // get the value of x or 25 if x is null
```